Evaluation of Segmentation Performance on Existing and Self-Created Models

Jia Xuan Ng
psyjn3@nottingham.ac.uk

26 April 2024

Abstract

This paper evaluates the performance of two models in segmenting colour photographs into two classes: flower and background. One is a pre-existing model ResNet-18[1] paired with DeepLabv3+[2] architecture; and a self-designed model using U-Net[3] architecture. The method couples two major ideas: image augmentation methods and pixel label correction through the use of morphological image transforms(dilation, erosion, etc.). Segmentation is evaluated using the Oxford Flower Dataset[4] with 1360 images.

1 Introduction

With recent improvements in deep learning, semantic segmentation has come a long way; with U-Net(named after its shape) being developed in 2015 by Olaf Ronneberger, Philipp Fischer, and Thomas Brox for the use of segmenting biomedical images, replacing FCN[5]. However soon after DeepLabv1 burst into the scene, a novel approach of pairing DCNNs with atrous(also known as dilated) convolutions and CRFs[6]. The idea of atrous convolutions is to expand the field of view while minimising the cost, and allowing a lessened use of aggressive up-sampling[7]. DeepLabv2 was presented in 2017, with the architecture almost remaining the same, apart from atrous convolutions evolving into Atrous Spatial Pyramid Pooling(ASPP) instead[7]. Soon to be was DeepLabv3, which appeared in December 2017, and aimed at improving the performance of previous approaches. What remained in the architecture was the DCNNs and the ASPPs, with the CRFs being removed, to allow end-to-end learning[8]. Surprisingly enough, although the smoothing layer was gone, performance surpassed previous architectures. Finally, DeepLabv3+ was presented in 2018, with the architecture being changed to encoder-decoder. Encoding was similar to the previous version, aside from the change to Separable Atrous Convolution instead of the original; this separated convolutions into 2 steps, Depthwise and Pointwise convolution[2]. Depthwise convolution applies different filters to each channel, whilst Pointwise(1x1) convolution outputs depthwise convolutions across said channels. Decoding is still quite simple, mainly to recover objects by up-sampling. In this paper the aim is to determine the performance in the two models, and whether certain augmentations can be used to improve performance or if it caused performance issues instead. The challenge is the small dataset provided, hence without adequate augmentation, over-fitting is very likely to occur. However, there is a fine line to balance between augmenting data to provide variability and over-augmenting and causing spatial information loss.

This paper is organised as so, with Section 2 describing the methodology used for pre-processing data, retraining the pre-existing model, and creating a U-Net model; along with the reasoning behind each, in an experimental fashion.

Section 3 will describe the evaluation process and the quality of segmentation against ground truth. It can be ambiguous to determine a "good" model against a "bad" one. One can say that a good model should segment the flower and the background from all the other classes; but does that demonstrate good thought practice? It can instead be posed as so; "Can the models segment the desired classes with the least error segmentation across other classes?".

In the end, Section 4 will be the conclusion to the paper, ending with a bigger viewpoint of the subject at hand, and whether it can be agreed that the topic can be closed or potential improvements can be suggested to be followed in the future.

2 Methodology

2.1 Overview

After loading the training images and labels, training images without corresponding label files are removed from use through the use of simple set operations regarding filenames. The images and labels are combined and shuffled, and then using simple percentage index calculations the size of the training, validation and testing sets can be derived. With that, the usable data length became about 850, requiring actions to be made to increase the amount that can be used to train the model.

2.2 Fixing Label Data

As mentioned prior, the ground truth(pixel labels) that was given to us had 5 classes: **null/boundaries**, **flower**, **leaves**, **background** and **sky**; with the convenient IDs ranging from **0-4**. However, the task at hand was simply to segment a colour image into the flower and the background classes. Hence, something had to be done, as purely training the model on said images, even with data augmentation, would produce a segmentation of other classes.

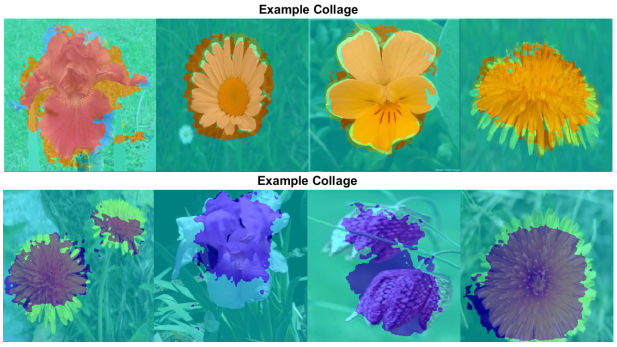

Figure 1: Segmentation results with all 5 classes(top) against only the **flower** and **background** classes(bottom) on ResNet-18 with DeepLabv3+ architecture with no augmentation. As seen, detail is lost because no labels have been fixed and if only segmenting with two classes all of the labels that are missing will be **undefined**, which is what we want to avoid.

The initial idea was to simply fill those missing labels using the background category. This means edges were lost as they are part of the null/boundaries class, making this naive. Subsequently, the idea was to use missing value filling algorithms like *knn*, *movmean* or *movmedian*. This required the conversion of categorical data into numerical as required by the algorithms; then turning it back to categorical data. This was time-consuming and could easily result in poor segmentation due to adding noise to the images, causing splashes of misclassified segmentation throughout the final image. The final decision was to apply metamorphic transformations(dilation, erosion, etc) with all 5 classes.

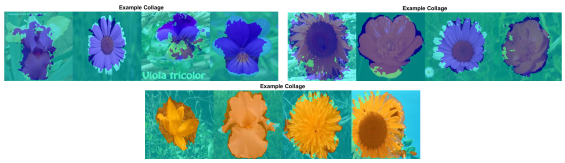

Figure 2: Segmentation results with 2 classes and **naive** label fix(first), 2 classes with numeric label fix using **knn** and filling the rest(second) and 5 classes with label fix using **dilation**(third).

Note that all cases are tested at **25 epochs**, hence dilation hasn't truly been given a chance to shine; as it has the best improvement rate of all the models. Generally, given long enough, it has better performance at segmentation: naive label fix having segmentation accuracy of **97.3%** for background and **73.4%** for flower, numeric label fix(where the categoric data is converted into numerical and algorithms applied to it) having an accuracy of **92.7%** and **87.9%** accuracy in the same order, and dilation label fix having an accuracy of **90.1%** and **91.9%**.

2.3 Data Augmentation

Data augmentation is the process of applying transformations to the data to introduce variability. It is a balancing act, as over-augmentation easily leads spatial information to being lost, producing less accurate models. The first experimentation was with random 2D affine; such as rotation, scale, translation, reflection, shear and cropping. Following this was smoothing(gaussian, box, median and anisotropic) and sharpening. Finally, additional ones explored were applying enhancements such as hue, saturation, brightness, and contrast. and ended off with contrast enhancements. The decision to avoid noise added that it would be counterproductive to the task at hand.

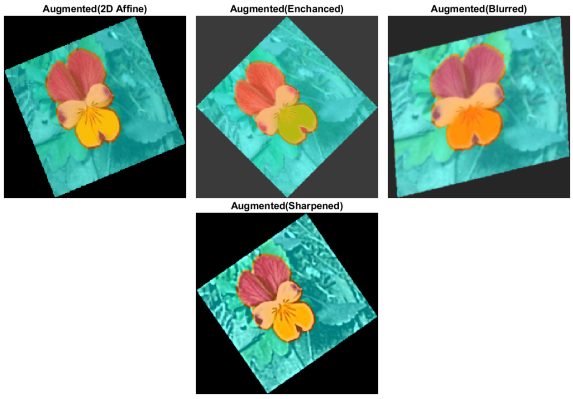

Figure 3: Images after applying random transformations, enhancement, blurring and sharpening(left to right).

2.4 ResNet-18 with DeepLabv3+ Architecture

The reason to pick ResNet-18 even though it isn't meant for segmentation was because of its efficacy. Furthermore, ease of use was another point of using the two together, as well as providing good results. There could have been choices made about adding/removing layers, but the thought followed was that adding and/or removing layers should be the last option taken; training options should be tweaked, similar to concepts in photography - where increasing brightness comes at modifying shutter speed, aperture and finally if no improvements, the ISO. All that was required to create the network was adding a pixel classification layer. Experimentation at varying epochs was done, from 10, 25, 50 and finally in the hundreds. The reason for this was to see where the model would start over-fitting or plateau; results were shocking as the epoch numbered near 450 when it finally started to plateau; where it ended with a **validation accuracy** of **81.09%**, **training accuracy** of about **93%** and a **loss** of about **0.7**. If the model was allowed to train further, the loss would have decreased more. To conclude this section, the training options optimised for the pre-existing model are: *Initial Learning Rate*: **1e-5**, *L2 Regularization*: **1e-2**, *Mini Batch Size*: **32**, *Validation Frequency*: **25**.

2.5 Self-Designed Model with U-Net Architecture

The reason for designing a model with the U-Net architecture was because of ease of use, efficacy, and strong results even with a shallow network. The model's background comes from biomedical imagery, but it's still one of the most used architectures that people creating models use. Doubly convolution layers were used to introduce non-linearity to enforce the self-designed model to learn complex features rather than remembering the input it gets. Following that comes a batch normalisation layer followed by a relu layer. After each doubly convolution layer comes to a max pooling layer, and this pattern occurred for 4 times, with the 4th having a dropout layer with a

50% drop rate before the max pooling, and the final being a doubly convolution layer followed by another dropout layer with a 50% drop rate(with no pooling). That sums it up for the encoder side of things, with the decoder layers being: a pattern of a transposed convolution layer, followed by batch normalisation, concatenation(inputs being the convolution layers on the encoder side and the transposed convolution layers from the decoder side), a doubly convolution layer with the usual batch normalisation layer and a relu layer. This pattern occurs 4 times, and the final layers are simply a 1x1 convolution layer followed by a softmax layer, and finally a pixel classification layer(output). In short, it downsamples the input consecutively then learns from it, and then upsamples the image without being too aggressive(hence the doubly convolution layers). To conclude this section, the training options optimised for the self-designed model are: *Initial Learning Rate*: **1e-4**, *L2 Regularization*: **1e-2**, *Mini Batch Size*: **32**, *Validation Frequency*: **25**. The initial test was to use the same settings, but given an initial learning rate of < 0.4, the model would underfit.

3 Evaluation

To evaluate each model segmentation, multiple metrics can be used to:*MeanIoU(Jaccard Index)*, *GlobalAccuracy*, *MeanAccuracy*, *WeightedIoU*, *MeanBFScore*. Given a 10% testing set, of the two models(pretrained and self-designed), we can see the difference in metric scores:

Global Accuracy	Mean Accuracy	MeanIoU	WeightedIoU	Mean BF Score
0.82832	0.38721	0.31186	0.69984	0.44303
0.75943	0.36461	0.27489	0.61236	0.29295

Table 1: Metrics between two models, pre-existing(top row) and self-designed(bottom-row). The pre-existing model has better scores as it trained to a higher number of epochs without plateauing or overfitting, unlike self-designed.

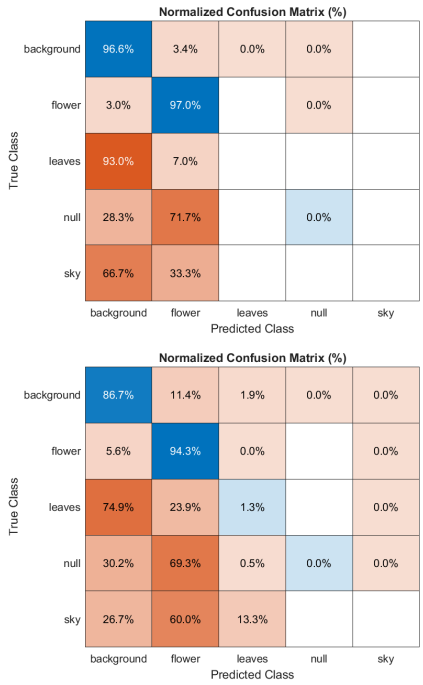

Figure 4: Two confusion matrices(normalised on row), pre-existing model(top) against self-designed(bottom). The pre-existing model segments better, with less false positive error, although the two models' true positives are roughly similar. This coupled with the fact the pre-existing model has no misclassifications in the unneeded classes, shows that it is a more accurate model.

4 Conclusion

It has been demonstrated that transfer learning using a pre-existing model(with deep learning instead of shallow learning) is much better than simply using encoder-decoder concepts without coupling it with additional algorithms. In hindsight re-feeding e.g. segmented images without labels, could be fed back to training, creating a feedback loop. Furthermore, additional tests could be made using lower mini-batch sizes such as < 16 to increase the model's ability to generalise. Another ending idea is to lower the false positives, employing techniques such as post-processing techniques for the model.

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[2] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European conference on computer vision (ECCV)*, pages 801–818, 2018.

[3] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.

[4] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, Dec 2008.

[5] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation, 2015.

[6] Shuai Zheng, Sadeep Jayasumana, Bernardino Romera-Paredes, Vibhav Vineet, Zhizhong Su, Dalong Du, Chang Huang, and Philip H. S. Torr. Conditional random fields as recurrent neural networks. In *2015 IEEE International Conference on Computer Vision (ICCV)*. IEEE, December 2015.

[7] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs, 2017.

[8] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation, 2017.